FastMap

by Gokhan Ceylan

Shape Rotator Virtual Hackathon

(Cryptographic Primitives & Identity)

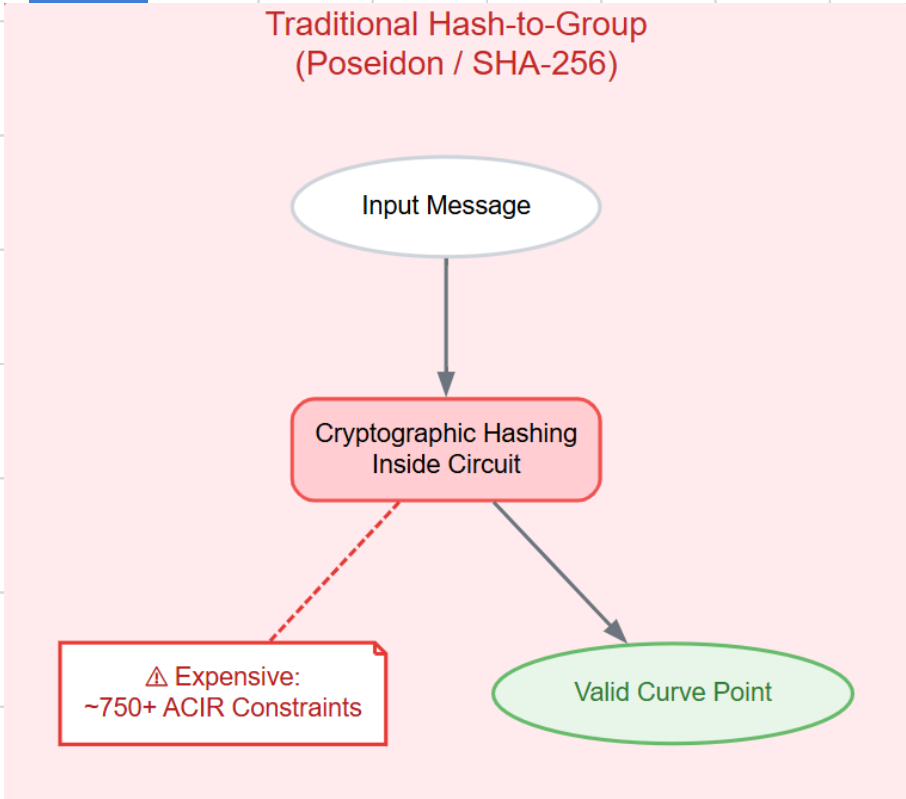
(Constraint-Friendly Hash-to-Group)

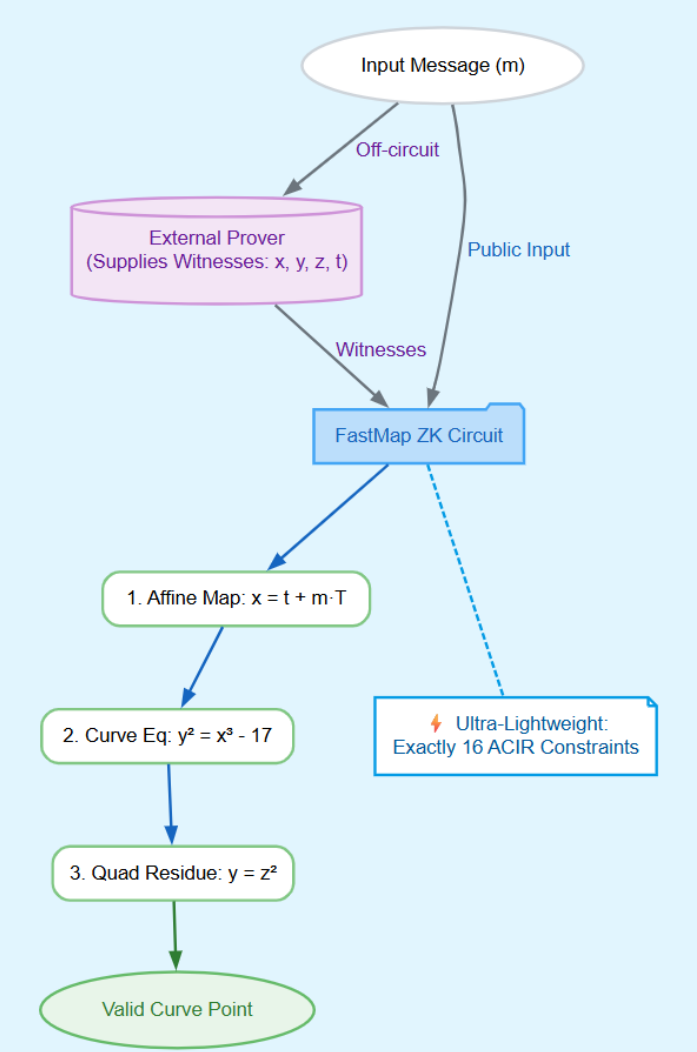
Problem

ZK proof systems require a hash-to-group operation constantly, in zkVMs, light clients, and signature verification. The best available methods cost 750+ constraints (Poseidon) or thousands (SHA-256). This bottleneck slows proving time and increases cost at scale. Every memory access in a zkVM needs this operation. Every BLS signature verification needs this operation. At production scale, this adds up to millions of expensive constraint evaluations per second.

Solution

FastMap implements the constraint-friendly map-to-elliptic-curve-group relation from Groth et al. (IC3, 2025). Using an increment-and-check approach with a quadratic residue check, we reduce the constraint cost to just 16 ACIR opcodes, proven secure in the Generic Group Model. This is a 47x improvement over Poseidon and over 50x over SHA-256. The library is written in Noir, runs client-side via Barretenberg WASM, and ships with a live interactive prover that anyone can run in their browser today.


Traditional Hash-to-Group
(Poseidon / SHA-256)
Input Message
Cryptographic Hashing
Inside Circuit
⚠ Expensive:
~750+ ACIR Constraints
Valid Curve Point


Input Message (m)
Off-circuit
External Prover
(Supplies Witnesses: x, y, z, t)
Public Input
Witnesses
FastMap ZK Circuit
1. Affine Map: x = t + m·T
2. Curve Eq: y² = x³ - 17
Ultra-Lightweight:
Exactly 16 ACIR Constraints
3. Quad Residue: y = z²
Valid Curve Point

Demo

https://fast-map-lemon.vercel.app/

Homepage

FastMap

Benchmarks Live Demo

Run a Proof →

Shape Rotator Virtual Hackathon

Making ZK Proofs 47× More Efficient

We implement the constraint-friendly map-to-elliptic-curve-group relation from Groth et al. | Replacing expensive hash-to-group functions with a 16-constraint alternative proven secure in the Generic Group Model.

16	750	47×
OUR CONSTRAINTS	POSEIDON CONSTRAINTS	REDUCTION VS POSEIDON

Simple Mode

YOUR MESSAGE

hello world

"hello world" "Ethereum block 21000000" "zkVM state root"

Generate ZK Proof

Proof verified | 403 ms

✓ 0x180f9b3c18992c0ea0724b227fa3bc2ed4f96bf17006b94c...

What was proved

Message "hello world" was hashed to a BN254 field element, a valid Grumpkin curve point was found in-browser via increment-and-check, and a ZK proof of the mapping was generated client-side using only **16 constraints**.

barretenberg-prover : zsh

```
$ Hashing message : BN254 scalar field element...
$ Hash of "hello world" : m = 18149469767584732552991 8…
$ Found valid point : t = 3
$ x = 59567716717651864496784044…
$ y = 16817564007361969760373042…
$ z = 12003159391140038131383089…
$ Passing computed parameters to ZK circuit...
$ Initializing Barretenberg WASM backend...
$ Executing circuit : generating witness...
$ Calling Barretenberg prover...
$ Proof generated in 403 ms
$ 0x180f9b3c18992c0ea0724b227fa3bc2ed4f96bf17006b94c8a49...
$ Verifying proof...
$ Proof verified ✓
```

Advanced Mode

Use Example Values

Example: message 7 maps to a verified Grumpkin point ($Y^2 = X^3 | 17$)

MESSAGE (M)

7

CURVE POINT X

1792

TWEAK INCREMENTS (T)

0

CURVE POINT Y

22226035320148080612132222638456355741437357070391599304795259579845819586 99

QUADRATIC RESIDUE ROOT (Z)

27675969817384653582149586519327496768237703087137381776462022467769682018 9

barretenberg-prover : zsh

```
$ Initializing Barretenberg WASM backend...
$ Executing circuit : generating witness...
$ Calling Barretenberg prover...
$ Proof generated in 389 ms
$ 0x05deb6a575761a55b855e181c4c3d595ffe58b7858d2a9cd5fd0...
$ Verifying proof...
$ Proof verified ✓
```

Proof verified | 389 ms

✓ 0x05deb6a575761a55b855e181c4c3d595ffe58b7858d2a9cd...